

LAPORAN WORKSHOP DevOps

Bab 3 - Docker Network, Volume, Bind Mount, tmpfs, dan Compose



Nama : Fitria Sari Ayuningtyas

NRP : 3126640013

Dosen Pengajar : Dr. Ferry Astika Saputra, ST, M.Sc

**PROGRAM STUDI TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA (PENS)
TAHUN 2026**

1. Tujuan

- Memahami komunikasi antar-container menggunakan user-defined bridge.
- Membandingkan named volume, bind mount, dan tmpfs.
- Menerapkan Docker Compose untuk aplikasi multi-container.
- Melakukan verifikasi dan troubleshooting container.

2. Lingkungan Praktikum

Komponen	Spesifikasi
OS	Ubuntu pada WSL2
Docker Engine	29.7.2
Docker Compose	v5.5.0
Web Server	Nginx Alpine
Backend	Flask
Database	PostgreSQL 16

3.

3.1 User-defined Bridge Network

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3$ docker network inspect lab-net
[
  {
    "Name": "lab-net",
    "Id": "4aa5180410859ac670e737636a335db132f2708e12c177388876b4ba580e2d6d",
    "Created": "2026-09-11T14:28:19.818206707+07:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": true,
    "EnableIPV6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.20.0.0/16",
          "Gateway": "172.20.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {},
    "Labels": {},
    "Containers": {
      "270803065da5579ffe34b3cfb293e358148a82ed78269fa118bc640cc1c66e72": {
        "Name": "server-b",
        "EndpointID": "386924d658caff060201e449097935740e3b34949697b3a6b85f4b8ecf98d98c",
        "MacAddress": "92:bb:7a:49:f2:55",
        "IPv4Address": "172.20.0.3/16",
        "IPv6Address": ""
      },
      "375710cae01e47605af43db8e1c885bc8b4277d55dc80bda475b614696345ffb": {
        "Name": "server-a",
        "EndpointID": "886dc4800ad070a89b8dd93bd5c1bd8f3d1c0cbb4e2a86b610fd3941c43fb65",
        "MacAddress": "7e:3d:de:bb:e8:3c",
        "IPv4Address": "172.20.0.2/16",
        "IPv6Address": ""
      }
    },
    "Status": {
      "IPAM": {
        "Subnets": [
          {
            "172.20.0.0/16": {
              "IPsInUse": 5,
              "DynamicIPsAvailable": 65531
            }
          }
        ]
      }
    }
  }
]
```

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-b
PING server-b (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.997 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.149 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.127 ms

--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.127/0.424/0.997 ms
```

Analisis

User-defined bridge network lab-net berhasil menghubungkan container server-a dan server-b. Pengujian menggunakan nama container server-b berhasil melakukan resolusi nama menjadi alamat IP 172.20.0.3 dengan 0% packet loss. Hal ini menunjukkan bahwa container pada user-defined bridge dapat saling berkomunikasi menggunakan nama.

3.2 Named Volume

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Fri Sep 11 08:31:33 UTC 2026
Fri Sep 11 08:31:38 UTC 2026
Fri Sep 11 08:31:43 UTC 2026
Fri Sep 11 08:31:48 UTC 2026
Fri Sep 11 08:31:53 UTC 2026
Fri Sep 11 08:31:58 UTC 2026
Fri Sep 11 08:32:03 UTC 2026
Fri Sep 11 08:32:08 UTC 2026
Fri Sep 11 08:32:13 UTC 2026
Fri Sep 11 08:32:18 UTC 2026
Fri Sep 11 08:32:23 UTC 2026
Fri Sep 11 08:32:28 UTC 2026
Fri Sep 11 08:32:33 UTC 2026
Fri Sep 11 08:32:38 UTC 2026
Fri Sep 11 08:32:43 UTC 2026
Fri Sep 11 08:32:48 UTC 2026
Fri Sep 11 08:32:53 UTC 2026
```

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3$ ls -lh data-vol-backup.tar.gz
-rw-r--r-- 1 root root 1.2K Sep 11 16:19 data-vol-backup.tar.gz
asus@LAPTOP-OC46B530:~/docker-lab/bab-3$
```

Analisis

Data pada named volume data-vol tetap tersedia setelah container writer dihapus. File log.txt masih dapat dibaca melalui container baru yang menggunakan volume tersebut. Proses backup juga berhasil menghasilkan data-vol-backup.tar.gz, sehingga named volume dapat digunakan untuk penyimpanan data yang persisten dan proses pencadangan.

3.3 Bind Mount

```
asus@LAPTOP-OC46B530:~$ docker exec brave_davinci cat /usr/share/nginx/html/index.html
Hello from host!
asus@LAPTOP-OC46B530:~$ docker exec brave_davinci cat /usr/share/nginx/html/index.html
Hello after host change!
```

Analisis

Bind mount memungkinkan direktori pada host dipetakan langsung ke dalam container. Setelah isi index.html pada host diubah, perubahan tersebut dapat langsung dibaca dari dalam container tanpa melakukan rebuild image. Hal ini menunjukkan bahwa bind mount cocok digunakan ketika container perlu mengakses file yang dikelola langsung dari host.

3.4 Tmpfs

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$ docker exec tmpfs-restart cat /app/tmp/test.txt
data sementara
asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$ docker restart tmpfs-restart
tmpfs-restart
asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$ docker exec tmpfs-restart ls -la /app/tmp
total 4
drwxr-xr-x  2 root  root   40 Sep 11 11:47 .
drwxr-xr-x  3 root  root  4096 Sep 11 11:46 ..
asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$
```

Analisis

File test.txt berhasil dibuat dan dibaca sebelum container di-restart. Setelah restart, file tersebut tidak ditemukan pada direktori /app/tmp. Hasil ini menunjukkan bahwa tmpfs menyimpan data secara sementara dan tidak mempertahankan data setelah container restart.

3.5 Compose Multi-container

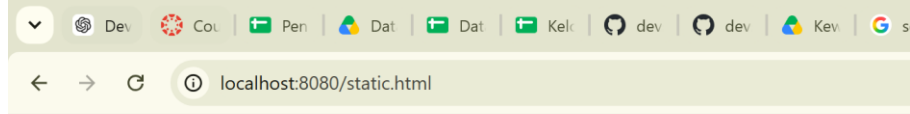
Docker Compose digunakan untuk menjalankan tiga service, yaitu Nginx sebagai web server, Flask sebagai backend, dan PostgreSQL sebagai database. Service web berada pada network frontend, sedangkan app terhubung ke frontend dan backend. PostgreSQL berada pada backend dan menggunakan named volume pg-data untuk penyimpanan data.

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$ docker compose ps
NAME                IMAGE             COMMAND                  SERVICE    CREATED   STATUS    PORTS
compose-lab-app-1   compose-lab-app   "gunicorn --bind 0.0.0.0 ..." app        2 hours ago Up 2 hours 5000/tcp
compose-lab-db-1    postgres:16-alpine "docker-entrypoint.s..." db         12 days ago Up 2 hours (healthy) 5432/tcp
compose-lab-web-1   nginx:alpine      "/docker-entrypoint..." web        2 hours ago Up 2 hours 127.0.0.1:8080->80/tcp

asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$ curl -i http://localhost:8080/health
HTTP/1.1 200 OK
Server: nginx/1.31.5
Date: Wed, 23 Sep 2026 09:05:37 GMT
Content-Type: application/json
Content-Length: 21
Connection: keep-alive

{"status": "healthy"}
asus@LAPTOP-OC46B530:~/docker-lab/bab-3/compose-lab$ curl http://localhost:8080/static.html
```

```
asus@LAPTOP-0C46B530:~/docker-lab/bab-3/compose-lab$ curl http://localhost:8080/static.html
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx/1.31.5</center>
</body>
</html>
```



Docker Compose Bab 3

Bind mount Nginx berhasil digunakan.

Hasil docker compose ps menunjukkan bahwa ketiga service berhasil berjalan dan PostgreSQL berstatus **healthy**. Pengujian endpoint `http://localhost:8080/` menghasilkan response dengan status ok dan informasi versi PostgreSQL, yang menunjukkan bahwa Nginx berhasil meneruskan request ke Flask dan Flask berhasil terhubung ke PostgreSQL. Pengujian endpoint `/health` juga menghasilkan **HTTP 200 OK** dengan response `{"status":"healthy"}`. Pengujian halaman statis melalui `http://localhost:8080/static.html` berhasil menampilkan halaman HTML..

3. 6 Evaluasi dan Latihan Mandiri

1. Mengapa user-defined bridge lebih baik daripada default bridge?

User-defined bridge menyediakan name resolution antar-container secara otomatis dan memberikan konfigurasi network yang lebih terkontrol dibandingkan default bridge.

2. Apa risiko bind mount terhadap keamanan host?

Bind mount dapat memberikan container akses langsung ke file atau direktori host. Jika mount menggunakan mode writable atau direktori yang terlalu luas, container yang dikompromikan berpotensi mengubah atau merusak data host.

3. Apa perbedaan docker compose down dan docker compose down -v?

docker compose down menghentikan dan menghapus container serta network yang dibuat Compose, sedangkan docker compose down -v juga menghapus named volume yang digunakan oleh Compose. Karena itu, down -v dapat menyebabkan data persistent pada volume hilang.

4. Kapan depends_on dengan healthcheck lebih tepat?

depends_on dengan healthcheck lebih tepat ketika service bergantung pada service lain yang harus benar-benar siap digunakan, misalnya aplikasi Flask yang membutuhkan PostgreSQL sudah siap menerima koneksi.

5. Bagaimana strategi backup volume database produksi?

Backup dilakukan secara berkala menggunakan mekanisme yang konsisten dengan database, disimpan pada lokasi terpisah, dan diuji dengan proses restore secara berkala. Backup juga sebaiknya memiliki retensi dan perlindungan akses yang sesuai.

4. Analisa

- **Masalah dan Diagnosis**

Pada pengujian awal endpoint `http://localhost:8080/static.html`, Nginx mengembalikan response **404 Not Found**. Untuk mendiagnosis masalah tersebut, dilakukan pemeriksaan terhadap isi direktori html menggunakan perintah `ls -la html`. Hasil pemeriksaan menunjukkan bahwa pada direktori tersebut hanya terdapat file `index.html`, sedangkan konfigurasi Nginx mengarahkan endpoint `/static.html` ke file `static.html`. Masalah kemudian diperbaiki dengan menambahkan file `static.html` pada direktori `html`. Setelah dilakukan pengujian ulang, endpoint `/static.html` berhasil menampilkan halaman HTML.

```
msusLAPTOP-OCM68530:~/docker-lab/bab-3/compose-lab$ docker compose logs --tail 50
db-1 | 2026-09-11 11:26:12.143 UTC [41] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2026-09-11 11:26:12.156 UTC [41] LOG:  database system was shut down at 2026-09-11 11:26:11 UTC
db-1 | 2026-09-11 11:26:12.156 UTC [41] LOG:  database system is ready to accept connections
web-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
db-1 | done
db-1 | server started
db-1 | CREATE DATABASE
web-1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
web-1 | 10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf (read-only file system?)
db-1 |
web-1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
db-1 |
web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
web-1 | /docker-entrypoint.sh: Configuration complete; ready for start up
web-1 | 2026/09/23 09:03:10 [notice] 1#1: using the "epoll" event method
web-1 | 2026/09/23 09:03:10 [notice] 1#1: nginx/1.31.5
web-1 | 2026/09/23 09:03:10 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
web-1 | 2026/09/23 09:03:10 [notice] 1#1: OS: Linux 6.18.33.2-microsoft-standard-WSL2
web-1 | 2026/09/23 09:03:10 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:1048576
web-1 | 2026/09/23 09:03:10 [notice] 1#1: start worker processes
web-1 | 2026/09/23 09:03:10 [notice] 1#1: start worker process 21
web-1 | 2026/09/23 09:03:10 [notice] 1#1: start worker process 22
web-1 | 2026/09/23 09:03:10 [notice] 1#1: start worker process 23
web-1 | 2026/09/23 09:03:10 [notice] 1#1: start worker process 24
web-1 | 2026/09/23 09:03:10 [notice] 1#1: start worker process 25
web-1 | 2026/09/23 09:03:10 [notice] 1#1: start worker process 26
web-1 | 2026/09/23 09:03:10 [notice] 1#1: start worker process 27
web-1 | 2026/09/23 09:03:10 [notice] 1#1: start worker process 28
web-1 | 172.18.0.1 - - [23/Sep/2026:09:04:16 +0000] "GET / HTTP/1.1" 200 178 "-" "curl/8.18.0" "-"
```

- **Risiko Keamanan dan Operasional**

Penggunaan `bind mount` dapat menjadi risiko apabila container diberikan akses tulis terhadap direktori `host` yang terlalu luas. Jika container mengalami kompromi, data pada `host` yang ter-mount berpotensi diubah atau dihapus. Selain itu, password database pada file `Compose` masih ditulis secara langsung sehingga kurang aman apabila konfigurasi dibagikan atau digunakan pada lingkungan `production`.

- **Rekomendasi untuk production-like environment**

Untuk lingkungan `production-like`, credential database sebaiknya dikelola menggunakan `secrets` atau mekanisme `secret management` sehingga tidak ditulis langsung pada file `Compose`. `Bind mount` juga perlu dibatasi hanya pada direktori yang diperlukan dan menggunakan mode `read-only` apabila container hanya membutuhkan akses baca. Penggunaan `Gunicorn` dan user `non-root` yang sudah diterapkan perlu dipertahankan. Selain itu, versi image Docker sebaiknya dipin agar deployment lebih konsisten dan dapat direproduksi. Backup database juga perlu dilakukan secara berkala dan proses `restore` perlu diuji secara rutin.

5. Kesimpulan

Praktikum Bab 3 berhasil menerapkan `user-defined bridge network`, `named volume`, `bind mount`, `tmpfs`, dan `Docker Compose`. Hasil pengujian menunjukkan bahwa container pada `user-defined network` dapat berkomunikasi menggunakan nama container, `named volume` dapat mempertahankan data setelah container dihapus, `bind mount` dapat meneruskan perubahan file dari `host` ke container, sedangkan `tmpfs` tidak mempertahankan data setelah container di-restart.

Docker Compose berhasil menjalankan tiga service, yaitu Nginx, Flask, dan PostgreSQL. PostgreSQL berhasil mencapai status `healthy`, Flask berhasil terhubung ke database, dan Nginx berhasil meneruskan request ke aplikasi. Pengujian endpoint utama, `health check`, serta halaman statis juga berhasil dilakukan. Dari praktikum ini

dapat dipahami bahwa pemilihan network, media penyimpanan, dan konfigurasi multi-container perlu memperhatikan aspek keamanan, persistensi data, serta kemudahan operasional.